

Linux 智能学习平台技术实现 README

全部代码已开源至<https://github.com/Leopardxv/GONGCHUANG>

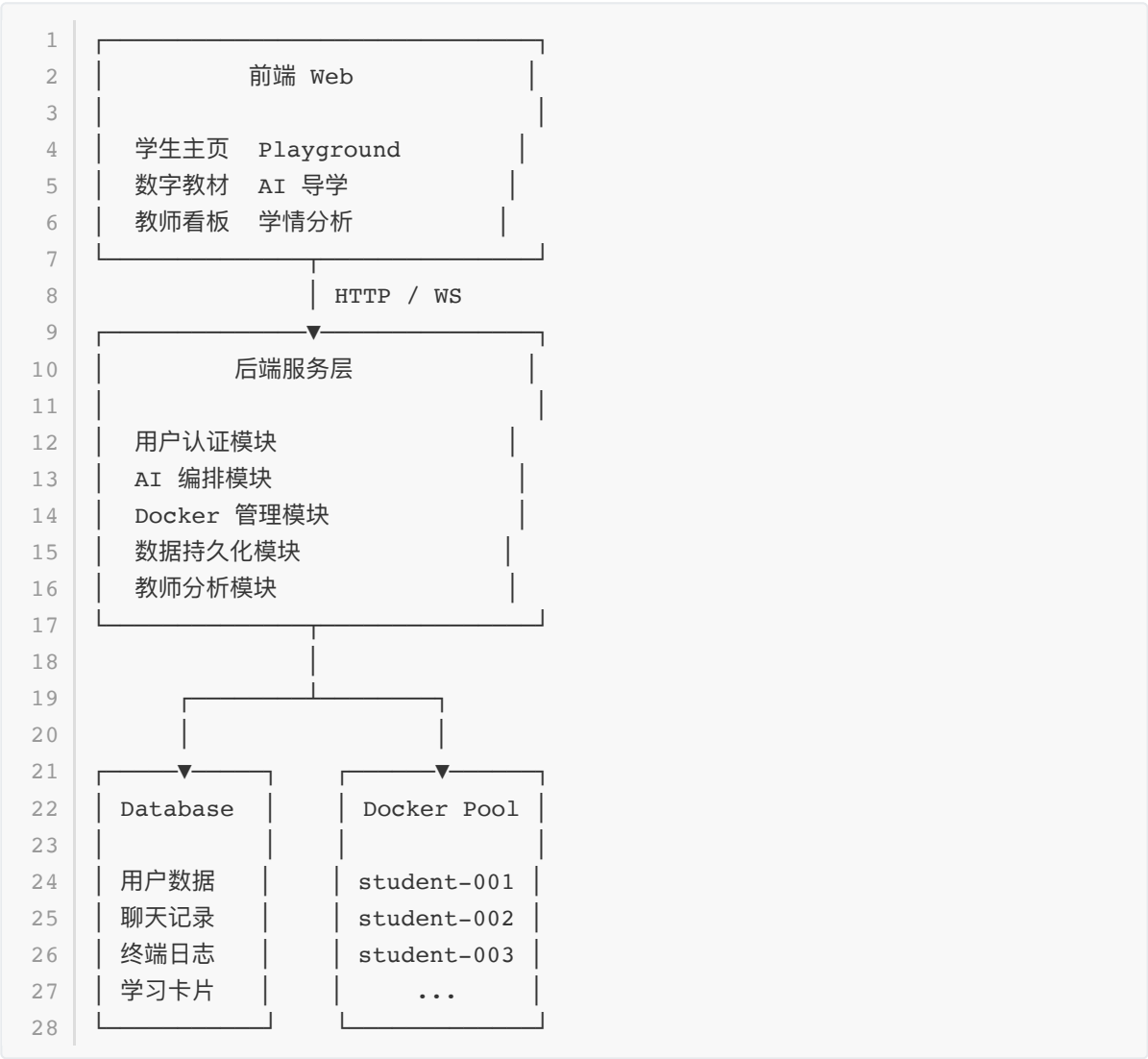
1. 项目概述

本项目面向 openEuler/Linux 实践教学场景，构建集数字教材、AI 导学问答、在线 Linux 实训、学习过程记录与教师学情分析于一体的智能学习平台。

系统不仅关注功能实现，更强调工程化设计，通过抽象编程、容器隔离和环境管理，实现平台的可扩展性、安全性与长期维护能力。

2. 整体架构设计

系统采用前后端分离架构：

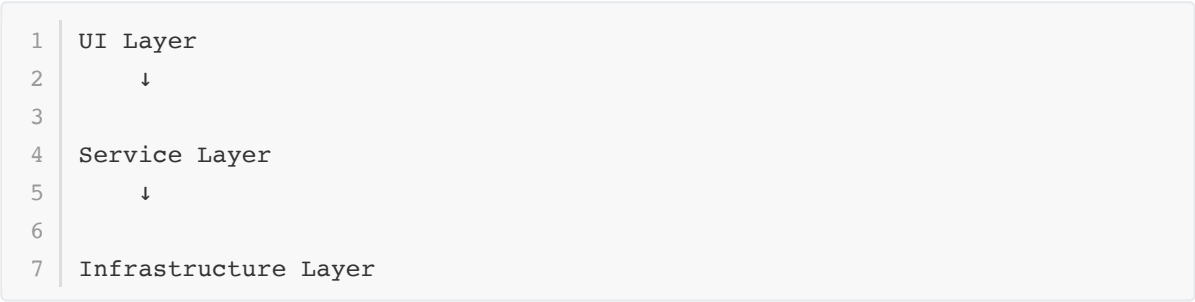


3. 抽象编程设计思想

3.1 核心原则

系统采用"高内聚、低耦合"的抽象设计。

所有功能均被抽象为独立模块：



页面只负责展示。

业务逻辑统一放入 Service。

底层资源统一由 Infrastructure 管理。

3.2 功能模块抽象

用户模块

负责：

- 1 登录
- 2 注册
- 3 权限校验
- 4 Token管理
- 5 角色管理

接口：

- 1 AuthService

AI 模块

负责：

- 1 主页问答
- 2
- 3 Playground命令分析
- 4
- 5 学习卡片生成

接口：

- 1 AIService

实现：

- 1 ChatTutor
- 2
- 3 CommandAnalyzer

后续可替换不同模型：

- 1 OpenAI
- 2
- 3 DeepSeek
- 4
- 5 Qwen API

无需修改业务代码。

Docker 模块

负责：

- 1 容器创建
- 2
- 3 容器查询
- 4
- 5 容器回收
- 6
- 7 容器状态监控

接口：

- 1 ContainerService

实现：

- 1 DockerManager

数据模块

负责：

1	聊天记录保存
2	
3	命令记录保存
4	
5	学习卡片保存
6	
7	教师统计分析

接口：

1	RecordService
---	---------------

3.3 抽象带来的优势

新增功能时：

1	新增AI模型
2	
3	新增教师分析功能
4	
5	新增终端环境

无需修改已有模块。

只需扩展对应 Service。

提高系统维护性。

4. 页面实现方式

4.1 学生主页

功能：

1	统一学习入口
2	
3	AI导学问答
4	
5	历史会话恢复

实现流程：

- 1 登录成功
- 2 ↓
- 3 获取用户信息
- 4 ↓
- 5 加载历史聊天
- 6 ↓
- 7 初始化主页状态

技术：

- 1 JWT认证
- 2
- 3 REST API
- 4
- 5 数据库持久化

4.2 数字教材页面

功能：

- 1 教材阅读
- 2
- 3 章节导航
- 4
- 5 学习进度记录

实现：

- 1 教材文件解析
- 2 ↓
- 3 目录树生成
- 4 ↓
- 5 阅读状态保存

技术：

- 1 Markdown/PDF解析
- 2
- 3 数据库记录

4.3 Playground

功能：

1	Linux命令执行
2	
3	实时输出
4	
5	学习卡片生成

流程：

1	用户输入命令
2	↓
3	WebSocket发送
4	↓
5	Docker Shell执行
6	↓
7	stdout/stderr返回
8	↓
9	异步调用AI分析
10	↓
11	生成学习卡片

核心原则：

1	终端优先
2	
3	AI并行

保证终端体验。

4.4 教师端

功能：

1	班级统计
2	
3	学习趋势
4	
5	错误分析
6	
7	学生画像

流程：

- 1 终端日志
- 2 ↓
- 3 行为统计
- 4 ↓
- 5 指标计算
- 6 ↓
- 7 图表展示

5. Docker 实训环境设计

这是整个系统最核心的工程实现。

5.1 为什么使用 Docker

传统方案：

- 1 所有学生共享服务器

问题：

- 1 互相影响
- 2
- 3 环境被污染
- 4
- 5 权限难控制
- 6
- 7 难以恢复

Docker方案：

- 1 每人独立Linux环境

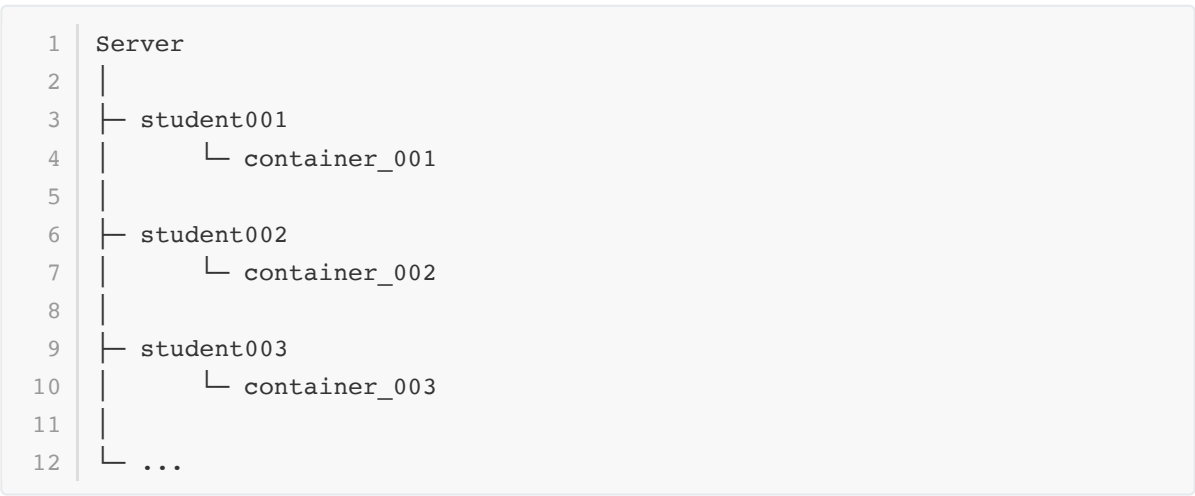
优势：

- 1 安全隔离
- 2
- 3 环境一致
- 4
- 5 快速创建
- 6
- 7 易于管理

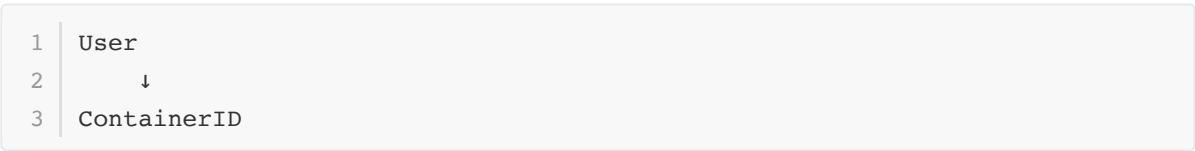
6. 每个学生 Docker 的组织方式

系统为每个学生分配专属容器。

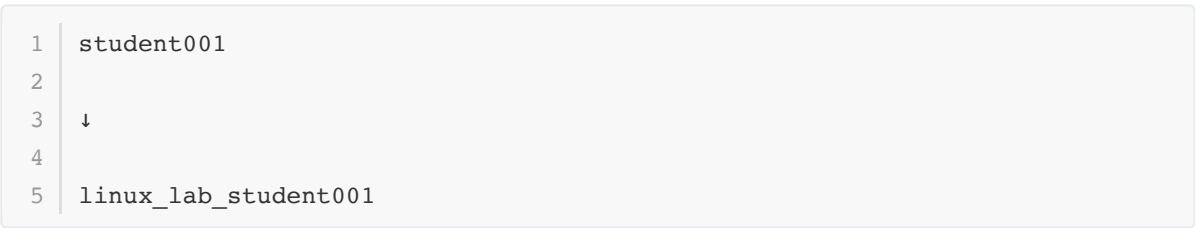
结构如下：



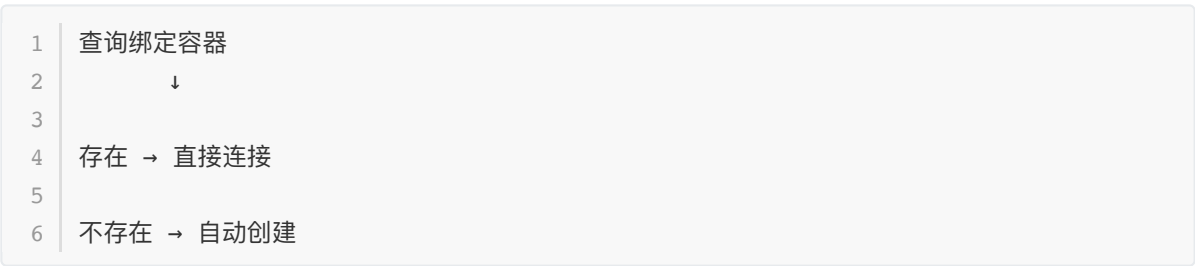
数据库记录：



例如：

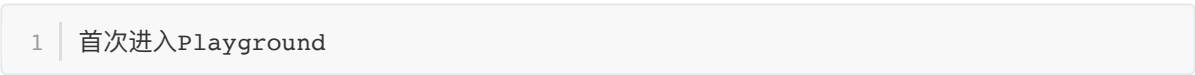


用户进入 Playground：



7. Docker 生命周期管理

容器创建：



容器运行：

- 1 | 学习过程中持续使用

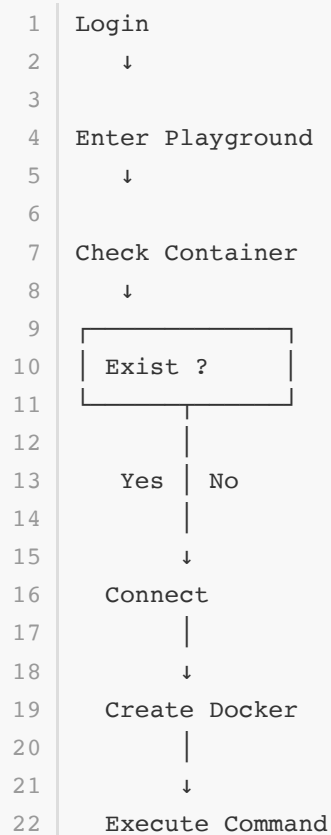
容器暂停：

- 1 | 用户退出登录

容器销毁：

- 1 | 课程结束
- 2 |
- 3 | 管理员清理

流程：



8. Docker 内部结构

每个容器内部：

- 1 | openEuler/Linux
- 2 | |
- 3 | └─ bash
- 4 | └─ 常用Linux命令
- 5 | └─ 教学测试文件
- 6 | └─ 学生工作目录

学生只能访问：

```
1 | /home/student
```

无法影响宿主机。

9. WebSocket 与 Docker 通信

连接关系：

```
1 | Browser
2 |   ↓
3 | WebSocket
4 |   ↓
5 | Backend
6 |   ↓
7 | Docker API
8 |   ↓
9 | Container Shell
```

实现效果：

```
1 | 浏览器 ≈ 本地终端
```

学生获得真实 Linux 操作体验。

10. Python venv 环境管理

开发阶段采用：

```
1 | venv
```

进行依赖隔离。

结构：

```
1 | project/
2 | |
3 | |─ backend/
4 | |─ frontend/
5 | |─ .venv/
6 | |─ requirements.txt
7 | └─ README.md
```

作用：

- 1 避免系统环境污染
- 2
- 3 避免依赖冲突
- 4
- 5 保证部署一致性

部署步骤：

```
1 python -m venv .venv
2
3 source .venv/bin/activate
4
5 pip install -r requirements.txt
```

通过：

```
1 requirements.txt
```

即可快速重建环境。

11. 核心工程特点

本项目不仅实现了 Linux 教学平台的功能需求，更注重工程化设计。

主要特点包括：

- 1 抽象编程设计
- 2 ↓
- 3 提高扩展能力
- 4
- 5 Docker 独立实验环境
- 6 ↓
- 7 保证教学安全与一致性
- 8
- 9 学生专属容器管理
- 10 ↓
- 11 实现真实在线实训
- 12
- 13 venv 环境隔离
- 14 ↓
- 15 提高开发与部署效率
- 16
- 17 WebSocket 实时通信
- 18 ↓
- 19 提供接近本地终端的体验

这些设计共同支撑了平台从课程演示系统向长期运行教学平台演进的可能性。